

divides the data into stratified training and test sets. Standardisation places both features on a comparable scale.

Encoding flower features into qubits

After preprocessing, petal length controls the rotation of Qubit 0 and petal width controls the rotation of Qubit 1. Different flowers therefore create different qubit states. The values are used as rotation instructions rather than being stored in the qubits as ordinary binary numbers.

Building the quantum circuit

The circuit first performs feature encoding, then applies trainable rotations and entanglement. Entanglement allows the circuit to process the relationship between the two flower measurements. Finally, both qubits are measured to produce two ordinary numerical outputs.

Connecting PennyLane with PyTorch

PennyLane's TorchLayer converts the QNode into a PyTorch-compatible layer. A classical linear layer receives the two quantum measurements, and a sigmoid output converts them into the probability of the flower being Versicolor.

Training the hybrid QML model

For every epoch, the model processes all training flowers, calculates prediction loss, and uses the Adam optimiser to update both the trainable quantum angles and the classical layer parameters. PennyLane supplies gradients through the circuit while PyTorch manages the optimisation loop.

Here's the complete Python Program:

```
import numpy as np
import pennylane as qml
import torch
from torch import nn
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import
train_test_split
from sklearn.preprocessing import
StandardScaler
from sklearn.metrics import accuracy_
score, confusion_matrix
```

```
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
```

```
# 1. Load two classes and two petal
features
iris = load_iris()
keep = iris.target < 2
X = iris.data[keep][:, [2, 3]]
y = iris.target[keep]
```

```
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.25, random_
state=SEED, stratify=y
)
```

```
# 2. Scale the data, then bound it into
convenient angles
scaler = StandardScaler()
X_train = np.arctan(scaler.fit_
transform(X_train))
X_test = np.arctan(scaler.transform(X_
test))
```

```
X_train = torch.tensor(X_train,
dtype=torch.float32)
X_test = torch.tensor(X_test,
dtype=torch.float32)
y_train = torch.tensor(y_train,
dtype=torch.float32).reshape(-1, 1)
y_test = torch.tensor(y_test,
dtype=torch.float32).reshape(-1, 1)
```

```
# 3. Create a two-qubit simulator
dev = qml.device("default.qubit",
wires=2)
```

```
# 4. Define the trainable quantum circuit
@qml.qnode(dev, interface="torch", diff_
method="backprop")
def circuit(inputs, weights):
    qml.AngleEmbedding(inputs, wires=[0,
1], rotation="Y")
    qml.BasicEntanglerLayers(weights,
```

```
wires=[0, 1])
    return [qml.expval(qml.PauliZ(w))
for w in [0, 1]]
```

```
# 5. Convert the QNode into a PyTorch
layer
quantum_layer = qml.qnn.TorchLayer(
    circuit, {"weights": (2, 2)}
)
```

```
model = nn.Sequential(
    quantum_layer,
    nn.Linear(2, 1),
    nn.Sigmoid()
)
```

```
# 6. Train quantum and classical
parameters together
loss_fn = nn.BCELoss()
optimizer = torch.optim.Adam(model.
parameters(), lr=0.05)
```

```
for epoch in range(1, 51):
    optimizer.zero_grad()
    probability = model(X_train)
    loss = loss_fn(probability, y_train)
    loss.backward()
    optimizer.step()
```

```
    if epoch == 1 or epoch % 10 == 0:
        pred = (probability >= 0.5).
int().numpy().ravel()
        acc = accuracy_score(y_train.
numpy().ravel(), pred)
        print(f"Epoch {epoch:02d}:
loss={loss.item():.4f}, "
            f"training
accuracy={acc:.3f}")
```

```
# 7. Evaluate unseen test flowers
model.eval()
with torch.no_grad():
    probability = model(X_test)
```

```
    prediction = (probability >= 0.5).int().
numpy().ravel()
    truth = y_test.int().numpy().ravel()

    print("Test accuracy:", accuracy_
score(truth, prediction))
    print("Confusion matrix:\n", confusion_
matrix(truth, prediction))
```